762. Prime Number of Set Bits in Binary Representation

Easy | Topics | Companies | Hint

Given two integers `left` and `right`, return *the **count** of numbers in the **inclusive** range* `[left, right]` *having a **prime number of set bits** in their binary representation.*

Recall that the **number of set bits** an integer has is the number of `1`'s present when written in binary.

- For example, `21` written in binary is `10101`, which has `3` set bits.

Example 1:

```
Input: left = 6, right = 10
Output: 4
Explanation:
6  -> 110 (2 set bits, 2 is prime)
7  -> 111 (3 set bits, 3 is prime)
8  -> 1000 (1 set bit, 1 is not prime)
9  -> 1001 (2 set bits, 2 is prime)
10 -> 1010 (2 set bits, 2 is prime)
4 numbers have a prime number of set bits.
```

Example 2:

```
Input: left = 10, right = 15
Output: 5
Explanation:
10 -> 1010 (2 set bits, 2 is prime)
11 -> 1011 (3 set bits, 3 is prime)
12 -> 1100 (2 set bits, 2 is prime)
13 -> 1101 (3 set bits, 3 is prime)
14 -> 1110 (3 set bits, 3 is prime)
15 -> 1111 (4 set bits, 4 is not prime)
5 numbers have a prime number of set bits.
```

Constraints:

- $1 \le left \le right \le 10^6$
- $0 \le right - left \le 10^4$

DS | Day-92 | 8-11-26 |

Leetcode 762 = Prime number of set Bits in Binary representation. __/__/____

We are given two integers:- • left • right
We need to count how many numbers in the inclusive range [left, right] have a prime number of set bits in their binary representation.

Set bits → a set bit is simply a bit whose value is 1 in binary representation. for ex = 21 = (10101)$_2$
there are 3 1's.
therefore, number = 21
binary = 10101
set bit = 3.

prime number → a number greater than 1 that has exactly two factors: 1 & itself.
some prime numbers:- 2 3 5 7 11 13 17 19.

what we are actually checking?
→ for every number between left & right:
→ convert the # into binary (conceptually).
→ count the number of 1's.
→ check whether that count is prime
→ if prime increase our answer.

example = left = 6 ; right = 10 ;] → given

Let's examine every number.

Number	Binary	Setbits	Prime?
6	110	2	✓
7	111	3	✓
8	1000	1	x
9	1001	2	✓
10	1010	2	✓

prime count = 1+1+0+1+1 = 4 , so, O/P = 4.

Optimised using Brian Kernighan's Algo:

Brian Kernighan's algo is:

n = n & (n-1)

this operation removes the rightmost set bit (the last 1 when reading from right to left).

Ex: 12 = 1101 , n-1 = 12-1 = 12 = 1100.

```
  1101
- 0001
------
  1100
```

now perform bitwise AND:

```
  1101
& 1100
------
  1100
```

the rightmost 1 is removed.

we continue doing the same thing until the number becomes zero,

12 & (12-1) = 1100

```
  1100
- 0001
------
  1000
```

```
  1100
  1000
------
  1000
```

1000, - 8 4 2 1

0000

total set bits = 3,

T.C = O(N(B + √B))

N = total no in the given range

B = no. of binary digits in a number

K = no. of set bits (1s) in a number.

- O(B) → for checking every binary digit.
- O(K) → removes one set bit in each iteration.

process checking takes atmost O(√B).

for one number,

O(K + √B)

since K cannot exceed B, the worst case becomes :-

O(B + √B).

we perform, N = right - left + 1 times

so, O(N(B + √B))

S.C = O(1), we only use a few variables, we don't create any additional arrays or ds.

therefore, O(1).

Now, there are 2 observations that makes this problem easier.

obs 1: we don't need to manually count set bits,
C++ provides a built-in function:-
__builtin_popcount(n);
↳ it returns the number of set bits in the binary representation of an integer.

obs 2: we don't need to repeatedly check whether a number is prime,
look at the constraints:-
$1 <= left <= right <= 10^6$
what's the max. no. of binary digits needed to represent a number upto 10^6?
↳ $2^{20} = 1048576$
therefore, numbers upto 10^6 requires at most 20 binary bits,
that means the no. of set bits can never exceed 20,
so, the only possible prime set-bit counts are:-
2, 3, 5, 7, 11, 13, 17, 19
We only need to determine primality for numbers from 0 to 20.
Instead of checking primality repeatedly, we can precompute it using the Sieve of Eratosthenes.
↳ it helps us find all the prime numbers upto a given limit.

we create an array:-
vector<bool> isPrime(21, true);

initially, we assume every number is prime,
but: isPrime[0] = false
isPrime[1] = false,
bcuz 0, 1 are not prime,
Now, we eliminate multiples of each prime number.

2 is prime → eliminate 4, 6, 8, 10
3 ... → ,, 6, 9, 12 ...
5 ... → ,, 10, 15, 20 ...

the remaining numbers are prime.

class Solution { public:
int countPrimeSetBits(int left, int right) {
vector<bool> isPrime(21, true);
isPrime[0] = false; isPrime[1] = false; } bcuz 0 & 1 are not prime.
start from 2 → 9 is bcuz.
for (int i = 2; i * i <= 20; i++) {
if (isPrime[i]) {

initially, we assume every number is prime

suppose i = 3, ← for (int j = i * i; j <= 20; j += i) {
multiple of 3 are, 6, 9, 12, isPrime[j] = false;
but 6 was already eliminated by 2, we can start directly from 3*3,
for 5, 10, 15, 20 have 1, 1, 2, 3, 4, so, 5*3 = 25,
so starting from i * i avoids unnecessary work.

int ans = 0; → initialize the answer, initially, no valid numbers have been found.
→ iterate through the given range.
for (int i = left; i <= right; i++) {
int bit = __builtin_popcount(i);
Get counts the number of 1s in the binary representation.
if (isPrime[bit]) { → check whether the set bit count is prime,
ans++; → increment ans.
}
}
return ans; → return the ans.
};

why $i * i <= 20$, $i <= \sqrt{n}$

for ex, $n = 20$,
$\sqrt{20} = 4.47$.
we only need to check, $i = 2, 3, 4$, bcz every composite number has atleast one factor less than or equal to its square root.

for 20,
$2 \times 10 = 20$,
so, when we process $i = 2$, we eliminate 20 bcz it is a multiple of 2,
we don't need to process 10 bcz atlast, every factor pair has atleast one factor less than or equal to $\sqrt{20}$.

T.C = O(n log log n) S.C = O(n).

↳ we create a boolean array.

↳ we eliminate multiples of each prime number,

→ for 2, we eliminate multiples of 2,
→ for 3, " " " " 3
⋮ — — so on,

the total no. of marking operations is approx,

$$n\left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \cdots\right)$$

the sum over prime numbers give O(n log log n) complexity.

dry run ⇒ left = 10; right = 15;

1st one sieve prepares the prime lookup table, prime numbers between 0 & 20;

2, 3, 5, 7, 11, 13, 17, 19,

now let's iterate through the range,

i	Binary	popcount(i)	isPrime[bit]	ans
10	1010	2	true	1
11	1011	3	true	2
12	1100	2	true	3
13	1101	3	true	4
14	1110	3	true	5
15	1111	4	false	5.

return ans;

O/p = 5.

Accepted 202 / 202 testcases passed

Akankshyaa submitted at Nov 08, 2026 09:27

Analysis | Solution

Save 41%

Big Diwali Dhamaka

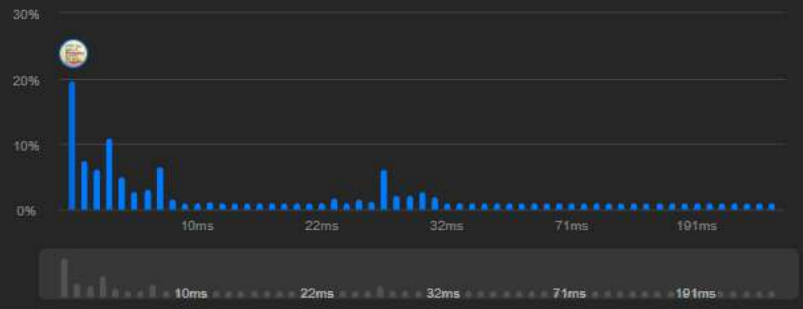
✨ Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

0 ms | Beats **100.00%**

Memory

7.90 MB | Beats **60.83%**



Code | C++

```cpp
class Solution {
public:
    int countPrimeSetBits(int left, int right) {
        vector<bool> isPrime (21,true);
        isPrime[0]=false;
        isPrime[1]=false;
        for(int i=2; i*i <= 20; i++){
            if(isPrime[i]){
                for(int j= i*i;j<=20;j+= i){
                    isPrime[j]=false;
                }
            }
        }
        int ans=0;
        for (int i=left;i<=right;i++){
            int bits=__builtin_popcount(i);
            if(isPrime[bits]){
                ans++;
            }
        }
        return ans;
    }
};
```

Saved

Testcase | Test Result

Accepted Runtime: 1 ms